Week 11 – Main Memory

Operating Systems Ch 9

Today's Schedule

Hour	Content
1st	**Quiz (Beginning)** → Theory Lecture (Part 1)
2nd	Theory Lecture (Part 2)
3rd	Hands-on Lab

Part 1

Background

Chapter Objectives

Topics covered in this chapter:

- Understand the difference between logical and physical addresses and the role of the MMU
- Allocate contiguous memory using First-fit, Best-fit, and Worst-fit strategies
- Distinguish between internal fragmentation and external fragmentation
- Perform logical-to-physical address translation in a paging system including TLB
- Describe Hierarchical, Hashed, and Inverted Page Table structures
- Understand address translation in IA-32, x86-64, and ARMv8 architectures

Why Memory Management Is Needed

To maximize CPU utilization, multiple processes must be loaded into memory simultaneously

- Storage directly accessible by the CPU: Register + Main Memory
- Data on disk must first be loaded into memory before the CPU can process it
- Since multiple processes share memory, management and protection are essential

```
+————————————+
| Register   |  ← Fastest (1 cycle)
+————————————+
|   Cache    |  ← Speed buffer (a few cycles)
+————————————+
|   Main     |  ← Direct CPU access (tens to hundreds of cycles)
|  Memory    |
+————————————+
|   Disk     |  ← Indirect access only (millions of cycles)
+————————————+
```

Basic Hardware

Storage that the CPU can directly access

- **Registers**: Inside the CPU, accessed in 1 clock cycle
- **Main Memory**: The only large-capacity storage the CPU can directly access
- **Cache**: Speed buffer between Register and Main Memory

Hardware for memory protection:

- **Base register**: The minimum physical address a process can access
- **Limit register**: The size of the address range
- If the CPU-generated address is out of range, a **trap** occurs

Base / Limit Register

```
   Memory Layout                  Hardware Protection
+──────────────────+
|  Operating System | (high)     CPU → address
+──────────────────+                    |
|                  |             +──────v──────+
|   Process P3     |             |  base ≤ addr  --no──→ TRAP
|                  |             |  < base+limit?        (illegal)
+──────────────────+ ← base+limit      |yes
|                  |             +──────v──────+
|   Process P2     |             |   Memory    |
|                  |
+──────────────────+ ← base (300040)
|   Process P1     |
+──────────────────+
|      (free)      | (low)
+──────────────────+
```

- If Base = 300040, Limit = 120900, then accessible range: 300040 ~ 420939
- Base/limit registers can only be modified in **kernel mode** (privileged instructions)

Address Binding

Classification based on when a program's addresses are determined

Timing	Description	Characteristics
Compile time	Memory start address fixed at compile time	Generates absolute code; recompilation needed if start address changes
Load time	Start address fixed at load time	Generates relocatable code
Execution time	Address binding performed during execution	Requires **MMU** hardware; used by most modern OSes

Multistep Processing of a User Program

```
source      compile       object       link        executable     load       program
program  ------------>    file     ------------>     file       ------------>  in memory
              time                     time                          time
            ^                        ^                             ^
        Compile-time            Load-time                   Execution-time
         binding                binding                      binding
                                  ^
                        dynamically linked
                           libraries
```

- Symbolic address -> Relocatable address -> Absolute address
- Address space mapping (binding) occurs at each stage

Logical vs Physical Address

Distinguishing two address spaces

- **Logical address** (= virtual address)
  - Address generated by the CPU
  - Address space from the process's perspective (0 ~ max)
- **Physical address**
  - Address actually delivered to the memory device
  - Address loaded into the Memory Address Register (R+0 ~ R+max)
- Compile-time / Load-time binding: logical = physical
- **Execution-time binding**: logical != physical (translation required)

MMU (Memory-Management Unit)

Hardware that translates logical addresses to physical addresses

```
Simplest MMU: Relocation Register scheme

+———————+   logical    +————————————+   physical    +———————+
|  CPU  | --346———→   |    MMU     | --14346———→   | Memory |
+———————+             | (relocation |               +———————+
                      |  reg=14000) |
                      |  346+14000  |
                      +————————————+
```

- User programs use only logical addresses (0 ~ max)
- The actual physical address is translated by the MMU
- The relocation register value is added to every logical address

Dynamic Loading

A technique that loads a routine into memory **only when it is called**

- Code that is not used is not loaded
- Useful when large programs have many rarely called routines (e.g., error handling)
- The total program size is large, but the actually used (loaded) portion is small
- Can be implemented by the **programmer without special OS support**

```
Main Program (in memory)
+-- routine_A() → already loaded
+-- routine_B() → loaded on call
+-- error_handler() → loaded only on error (on disk)
```

Dynamic Linking and Shared Libraries

A technique that links libraries **at execution time**

- **Static Linking**: Library included in the executable file (increases file size)
- **Dynamic Linking**: Links DLL/Shared Library at execution time

Advantages of DLL (Dynamic-Link Library) / Shared Libraries:

Aspect	Description
Memory savings	Only one copy of the library code in memory, shared by multiple processes
Disk savings	No need to copy the library into each executable
Easy updates	No recompilation needed when replacing the library
Version management	Compatibility maintained through version information

- **Stub**: A small piece of code that locates the library routine

Static Linking vs Dynamic Linking

```
Static Linking                      Dynamic Linking
+─────────────────+                 +─────────────────+
| Program A       |                 | Program A       |
| +─────────────+ |                 | (stub → libc)   |
| | libc copy 1 | |                 +─────────────────+
| +─────────────+ |                          |
+─────────────────+                 +─────────────────+
| Program B       |                 | Program B       |
| +─────────────+ |                 | (stub → libc)   |
| | libc copy 2 | |                 +─────────────────+
| +─────────────+ |                          |
+─────────────────+                 +─────────────────+
                                    | libc (1 copy)   | ← shared
                                    +─────────────────+

libc 2MB x 40 processes             libc 2MB x 1 copy
= 80MB memory usage                 = 2MB memory usage
```

Part 2

Contiguous Memory Allocation

Contiguous Memory Allocation Overview

A scheme that allocates **a single contiguous block of memory** to each process

- Memory is divided into OS area and user area
- Most OSes place the OS in high memory
- **Hole**: An available memory block (scattered in various sizes)
- When a process arrives, it is allocated to a hole of sufficient size

Memory Protection with Relocation

A protection mechanism combining Relocation register + Limit register

```
                    limit        relocation
                   register       register
                      |               |
                      v               v
CPU ⟶ logical ⟶ < ? --yes⟶ + ⟶ physical ⟶ Memory
        address        |                address
                      no
                       |
                       v
                   TRAP: addressing error
```

- **Limit register**: Restricts the range of logical addresses
- **Relocation register**: Added to logical address to produce physical address
- The dispatcher sets both register values appropriately during context switch

Variable Partition

A scheme that allocates variable-sized partitions to processes

```
Initial state      P8 exits        P9 allocated    P5 exits
+————————+     +————————+     +————————+     +————————+
|   OS   |     |   OS   |     |   OS   |     |   OS   |
+————————+     +————————+     +————————+     +————————+
|   P5   |     |   P5   |     |   P5   |     |  hole  |
|        |     |        |     |        |     |        |
+————————+     +————————+     +————————+     +————————+
|   P8   |     |  hole  |     |   P9   |     |   P9   |
|        |     |        |     +————————+     +————————+
+————————+     +————————+     |  hole  |     |  hole  |
|   P2   |     |   P2   |     +————————+     +————————+
+————————+     +————————+     |   P2   |     |   P2   |
                              +————————+     +————————+
```

- When a process terminates, a hole is created; adjacent holes are merged
- Problem: Which hole should a process be placed in?

Allocation Strategies: First-fit / Best-fit / Worst-fit

Dynamic Storage-Allocation Problem: Which hole should a request of size n be allocated to?

Strategy	Method	Characteristics
First-fit	Allocate to the first sufficient hole	Fast, generally good performance
Best-fit	Allocate to the smallest sufficient hole	Produces smallest remaining hole, requires full search
Worst-fit	Allocate to the largest hole	Produces largest remaining hole, requires full search

- Simulation results: First-fit and Best-fit outperform Worst-fit
- Between First-fit and Best-fit, First-fit is generally faster

Allocation Strategy Example

Memory holes: 300KB, 600KB, 350KB, 200KB, 750KB, 125KB Process requests: P1=115KB, P2=500KB, P3=358KB, P4=200KB, P5=375KB

```
First-fit:
  P1(115) → 300KB hole  |  P2(500) → 600KB hole
  P3(358) → 750KB hole  |  P4(200) → 200KB hole
  P5(375) → allocation failed (remaining holes: 185, 100, 392, 125)

Best-fit:
  P1(115) → 125KB hole  |  P2(500) → 600KB hole
  P3(358) → 750KB hole  |  P4(200) → 200KB hole
  P5(375) → 392KB hole

Worst-fit:
  P1(115) → 750KB hole  |  P2(500) → 635KB hole
  P3(358) → allocation failed
```

Fragmentation

Two types of memory fragmentation

External Fragmentation

- Total available memory is sufficient but **contiguous space is lacking**
- **50-percent rule**: When N blocks are allocated, 0.5N blocks become unusable due to fragmentation
  - Approximately 1/3 of total memory becomes unusable

Internal Fragmentation

- **Leftover space** because the request size is smaller than the allocation unit
- Example: Allocation unit 4KB, request 3.5KB -> 0.5KB wasted

Fragmentation Visualization

```
External Fragmentation              Internal Fragmentation
+————————+                          +————————+
|  P1    |                          |Allocated|  ← Request: 3.5KB
+————————+                          | (3.5KB)|
|  hole  | 50KB                     ├————————┤
+————————+                          | Wasted |  ← Internal fragmentation: 0.5KB
|  P2    |                          | (0.5KB)|
+————————+                          +————————+  ← Allocation unit: 4KB
|  hole  | 30KB
+————————+
|  P3    |        Request: 70KB
+————————+        Available: 50+30 = 80KB (sufficient)
|  hole  |        Contiguous: max 50KB (insufficient!)
+————————+
```

Compaction

Solution for external fragmentation

- Move used memory to one side to **create a large contiguous free space**
- Only possible with **execution-time binding** (addresses can be dynamically relocated)
- Very expensive (requires moving all processes + updating base registers)

```
Before Compaction              After Compaction
+--------+                  +--------+
|  P1    |                  |  P1    |
+--------+                  +--------+
|  hole  |                  |  P2    |
+--------+                  +--------+
|  P2    |                  |  P3    |
+--------+         →        +--------+
|  hole  |                  |        |
+--------+                  |  hole  | ← large contiguous space
|  P3    |                  |        |
+--------+                  +--------+
```

- A better alternative: **Paging** (non-contiguous allocation)

Part 3

Paging – Basic Method

Paging Overview

A technique that allocates a process's physical address space **non-contiguously**

- **Physical memory** -> divided into fixed-size **frames**
- **Logical memory** -> divided into same-size **pages**
- **Page table**: maps page number -> frame number
- **Completely eliminates** external fragmentation
- Internal fragmentation occurs only in the last page (average 0.5 page)
- Page size = Frame size (typically 4KB ~ 1GB, power of 2)
- The OS manages physical memory allocation state using a **frame table**

Paging – Address Translation Principle

Logical address structure (page size = 2^n, address space = 2^m)

```
Logical Address (m bits)
+——————————————————+——————————————————+
| Page number (p)   | Page offset (d)  |
|    (m-n bits)     |    (n bits)      |
+——————————————————+——————————————————+

Translation process:
1. Use p as the index into the page table
2. Obtain frame number f from the page table
3. Replace p with f → physical address

Physical Address
+——————————————————+——————————————————+
| Frame number (f)  | Page offset (d)  |
+——————————————————+——————————————————+
```

- Offset d **does not change** during translation

Paging Hardware

```
Logical Address                              Physical Address
+------+------+                              +------+------+
|  p   |  d   |                              |  f   |  d   |
+------+------+                              +------+------+
   |                                            ^
   v                                            |
+-----------------+                             |
| Page Table      |                             |
|  [0] → frame 5  |                             |
|  [1] → frame 6  |   p=1 → f=6 -------------+
|  [2] → frame 1  |
|  [3] → frame 2  |
+-----------------+
```

- When the CPU generates a logical address, the MMU consults the page table for translation
- Each process maintains a **separate page table**

Paging Example (32-byte memory, 4-byte pages)

```
Logical Memory          Page Table          Physical Memory
+-----+ page 0          0 → 5               +-----+ frame 0
| a b |                 1 → 6               |     |
| c d |                 2 → 1               +-----+ frame 1
+-----+ page 1          3 → 2               | i j |   ← page 2
| e f |                                     | k l |
| g h |                                     +-----+ frame 2
+-----+ page 2                              | m n |   ← page 3
| i j |                                     | o p |
| k l |                                     +-----+ frame 3
+-----+ page 3                              |     |
| m n |                                     +-----+ frame 4
| o p |                                     |     |
+-----+                                     +-----+ frame 5
                                            | a b |   ← page 0
logical addr 0 → phys 20 (5×4+0)           | c d |
logical addr 3 → phys 23 (5×4+3)           +-----+ frame 6
logical addr 4 → phys 24 (6×4+0)           | e f |   ← page 1
logical addr 13 → phys 9  (1×4+1)          | g h |
                                            +-----+ frame 7
```

Free-Frame Management

Frames are allocated from the free frame list when a process is assigned

```
(a) Before allocation                  (b) After allocation
free-frame list: 14,13,18,20,15        free-frame list: 15

+------+                             +------+
| ...  |                             | ...  |
+------+ frame 13                    +------+ frame 13
|      |  <----------+               |page 1|
+------+ frame 14     |               +------+ frame 14
|      |  <---------+ |               |page 0|
+------+            | |               +------+
| ...  |            | |               | ...  |
+------+ frame 18   | |               +------+ frame 18
|      |  <---+     | |               |page 2|
+------+      |     | |               +------+
| ...  |      |     | |               | ...  |
+------+ frame 20   | |               +------+ frame 20
|      |  <-+ |     | |               |page 3|
+------+    | |     | |               +------+
             | |     | |
new process page table:                page table:
page 0 -> 14                           0 -> 14
page 1 -> 13                           1 -> 13
```

Internal Fragmentation in Paging

Analysis of internal fragmentation in paging

- Page size = 2,048 bytes, Process size = 72,766 bytes
- Required pages: 72,766 / 2,048 = 35 pages + 1,086 bytes
- Allocation: **36 frames** -> Internal fragmentation = 2,048 - 1,086 = **962 bytes**
- Worst case: n pages + 1 byte -> n+1 frames allocated -> nearly 1 frame wasted

Trade-off when choosing page size:

Small page size	Large page size
Less internal fragmentation	More internal fragmentation
Larger page table	Smaller page table
More I/O required	More efficient I/O

Part 4

Paging – Hardware Support (TLB)

Hardware Implementation of Page Tables

Hardware approaches for accessing the page table

Method 1: Dedicated Registers

- Store the page table in high-speed hardware registers
- Advantage: Very fast translation
- Disadvantage: Only feasible when the page table is small (e.g., 256 entries)

Method 2: PTBR (Page-Table Base Register)

- Store the page table in **memory**, with PTBR pointing to its start address
- Only PTBR needs to be changed during context switch (fast switching)
- Problem: **Two memory accesses** required (page table once + data once)

TLB (Translation Look-aside Buffer)

A high-speed associative memory that serves as a **cache** for the page table

- Size: typically 32 ~ 1,024 entries
- **Key-Value** structure: page number -> frame number
- All keys compared **simultaneously** (parallel search)
- TLB lookup is part of the CPU instruction pipeline -> no performance penalty

```
TLB Hit/Miss process:

CPU → p,d → Search for p in TLB
              |
        +-----+-----+
        |           |
     TLB hit     TLB miss
        |           |
   f obtained    Fetch f from
   immediately   page table, then
        |        add (p,f) to TLB
        |           |
        +-----+-----+
```

TLB with Paging Hardware

```
Logical Address
+------+------+
|  p   |  d   |
+------+------+
   |
   v
+---------+       TLB hit
|   TLB   | ----------------> f --> +------+------+
| p  | f  |                         |  f   |  d   |  -> Physical Address
| 3  | 7  |       TLB miss         +------+------+
| 7  | 2  | ----------+                 |
| 1  | 5  |           v             +-------+
+---------+     +-------------+       | Memory |
                | Page Table  |       +-------+
                | [p] -> f    |
                +-------------+
                      |
                TLB update: add (p, f)
```

- When the TLB is full, existing entries are replaced (LRU, Round-Robin, Random)
- Some entries are **wired down** (for kernel code, non-replaceable)

ASID (Address-Space Identifier)

A scheme that includes a **process identifier (ASID)** in TLB entries

TLB without ASID	TLB with ASID
Entire TLB **flushed** on context switch	No flush needed on context switch
Prevents use of stale mappings from previous process	ASID distinguishes processes
Performance degradation (TLB miss surge at new process start)	TLB entries from multiple processes can coexist

```
TLB entry with ASID:
+------+------+------+
| ASID |  p   |  f   |
+------+------+------+
|  P1  |  3   |  7   |  ← Process 1's page 3 → frame 7
|  P2  |  3   |  12  |  ← Process 2's page 3 → frame 12
+------+------+------+
```

Effective Access Time (EAT)

Quantitative analysis of TLB performance

- **Hit ratio (alpha)**: Probability of finding the page number in the TLB
- **Memory access time**: Time for one memory access
- **TLB access time**: Time for one TLB lookup

EAT formula:

```
EAT = alpha x (TLB access + Memory access)
    + (1 - alpha) x (TLB access + 2 x Memory access)
```

EAT Calculation Examples

Example: Memory access = 100ns, TLB access = 10ns

Case 1: Hit ratio = 80%

```
EAT = 0.80 x (10 + 100) + 0.20 x (10 + 200)
    = 0.80 x 110 + 0.20 x 210
    = 88 + 42 = 130ns   (30% slowdown)
```

Case 2: Hit ratio = 98%

```
EAT = 0.98 x (10 + 100) + 0.02 x (10 + 200)
    = 0.98 x 110 + 0.02 x 210
    = 107.8 + 4.2 = 112ns   (approx. 2% slowdown)
```

Case 3: Hit ratio = 99% (realistic value)

```
EAT = 0.99 x 110 + 0.01 x 210
    = 108.9 + 2.1 = 111ns   (approx. 1% slowdown)
```

- The higher the hit ratio, the closer to raw memory access time!

Part 5

Paging – Protection & Shared Pages

Protection in Page Tables

Memory protection by adding **protection bits** to each page table entry

Protection Bits:

- **Read-only / Read-write** bit: Whether writing is allowed
- **Execute** bit: Whether code execution is allowed
- Violation -> hardware trap -> OS handles it

Valid-Invalid Bit:

- **Valid**: The page belongs to the process's logical address space
- **Invalid**: Access not allowed (page not in use)
- Accessing an invalid page -> trap to OS (invalid page reference)

Valid-Invalid Bit Example

14-bit address space (0~16383), page size = 2KB, program uses 0~10468

```
Page Table                  Physical Memory
+-------+-------+------+
| Page  | Frame | V/I  |     Process uses pages 0-5
+-------+-------+------+     (addresses 0 ~ 10468)
|   0   |   2   |  v   |
|   1   |   3   |  v   |     Page 5: addr 10240 ~ 12287
|   2   |   4   |  v   |     (but program only uses ~ 10468)
|   3   |   7   |  v   |     → addresses 10469~12287 in page 5
|   4   |   8   |  v   |       are accessible but not actually used
|   5   |   9   |  v   |       (internal fragmentation)
|   6   |   0   |  i   |     ← invalid: trap on access
|   7   |   0   |  i   |     ← invalid: trap on access
+-------+-------+------+
```

- **PTLR** (Page-Table Length Register): Limits the page table size to eliminate unnecessary entries

Shared Pages

Multiple processes running the same code **share the same frames**

Condition: The code must be **reentrant (read-only)**

```
Process P1 Page Table     Physical Memory     Process P2 Page Table
+---+---+                 +-----------+        +---+---+
| 0 | 3 |---> frame 3 <---| libc pg 1 |<-------| 0 | 3 |
| 1 | 4 |---> frame 4 <---| libc pg 2 |<-------| 1 | 4 |
| 2 | 6 |---> frame 6 <---| libc pg 3 |<-------| 2 | 6 |
| 3 | 1 |---> frame 1     | P1 data   |        | 3 | 7 |---> frame 7
+---+---+                 | ...       |        +---+---+    (P2 data)
                          +-----------+
```

- Example: Text editor used by 40 users
  - Shared: code pages (libc 2MB) x 1 copy = **2MB**
  - Individual: data pages x 40 copies
  - vs. without sharing: 80MB (40 x 2MB)

Shared Pages Details

Applications of shared pages

- **System libraries** (libc, DLLs): The most common sharing targets
- **Compilers, Editors, Window systems**: Used by multiple users simultaneously
- **Database systems**: Shared common code
- **Shared memory for IPC**: Also used for inter-process communication

Important notes:

- Shared code must be **read-only**
- The OS **enforces** the read-only property (using protection bits)
- Each process's data/stack pages are maintained individually

Part 6

Page Table Structure

The Problem of Large Page Tables

Page table size analysis for a 32-bit address space

```
32-bit logical address, 4KB page (2^12)
→ Page table entries = 2^32 / 2^12 = 2^20 = approx. 1,048,576
→ Each entry 4 bytes → Page table size = 4MB (per process!)

64-bit logical address, 4KB page
→ Page table entries = 2^64 / 2^12 = 2^52 ~ 4 x 10^15
→ Each entry 8 bytes → Page table size = 32PB (!!!)
```

- Difficult to place a 4MB page table contiguously in memory
- Solutions:
  1. **Hierarchical Paging** (multi-level paging)
  2. **Hashed Page Table**
  3. **Inverted Page Table**

Hierarchical Paging (Two-Level)

A technique that pages the page table itself

```
32-bit Logical Address (4KB page):

+──────────+──────────+──────────+
|    p1    |    p2    |    d     |
| (10 bit) | (10 bit) | (12 bit) |
+──────────+──────────+──────────+
     |          |          |
     v          v          v
Outer Page   Inner Page   Offset
  Table        Table     within Page
```

- p1: Index into the outer page table (1024 entries)
- p2: Index into the inner page table (1024 entries)
- d: Offset within the page (4096 bytes)
- Also called a **forward-mapped page table**

Two-Level Page Table Operation

```
Logical Address:  p1 | p2 | d

     p1                p2                  d
     |                 |                   |
     v                 v                   v
+----------+     +----------+         +-----------+
|          |     |          |         |           |
| Outer PT | →   | Inner PT | →       |   Frame   |
|  [p1]    |     |  [p2]=f  |        |  offset d |
|          |     |          |         |           |
+----------+     +----------+         +-----------+
     |                |                    |
     +---frame #----→-+----frame # + d ----+
                                |
                        Physical Address
```

Advantages:

- Inner page tables for unused address space regions are **not loaded into memory**
- Only the actually used portions consume memory, not the full 4MB

Limitations of Hierarchical Paging in 64-bit

Problems with two-level paging in a 64-bit address space

```
64-bit address, 4KB page, two-level:
+──────────+──────────+──────────+
|    p1    |    p2    |    d     |
| (42 bit) | (10 bit) | (12 bit) |
+──────────+──────────+──────────+

Outer page table = 2^42 entries = 2^44 bytes (16TB!)
→ Still too large

Three-level:
+───────+───────+───────+───────+
|  p1   |  p2   |  p3   |   d   |
| (32)  | (10)  | (10)  | (12)  |
+───────+───────+───────+───────+

Outer page table = 2^32 entries = 2^34 bytes (16GB!)
→ Still too large ... would need 7 levels → impractical
```

- Hierarchical paging is unsuitable for 64-bit systems
- Alternatives: **Hashed Page Table** or **Inverted Page Table**

Hashed Page Tables

A page table structure suitable for address spaces of 32 bits or more

```
Logical Address
+------+------+
|  p   |  d   |
+------+------+
   |
   v
Hash Function
   |
   v
+-----------------------------------------+
| Hash Table                              |
|  [h(p)] → (p, frame_r, next) →  ...     |
|                                         |
|  bucket: collisions resolved via        |
|          linked list                    |
+-----------------------------------------+
   |
   v
Obtain frame number r from the entry matching p
   |
   v
Physical Address = r | d
```

Inverted Page Tables

A structure that maintains a single global table based on **frames**

```
Logical Address
+------+------+------+
| pid  |  p   |  d   |
+------+------+------+
   |      |
   v      v
+---------------------------+
| Inverted Page Table       |    Number of entries = physical memory frame count
| [0] (pid_x, page_a)      |
| [1] (pid_y, page_b)      |
| ...                       |
| [i] (pid, p)  ← match!   |    → physical address = (i, d)
| ...                       |
| [N] (pid_z, page_c)      |
+---------------------------+
```

- Advantage: Significant memory savings (no per-process page table needed)
- Disadvantage: Slow search (full table scan required) -> mitigated with **hash table**
- Limitation: Shared pages are difficult to implement (1 physical page = 1 virtual mapping)

Page Table Structure Comparison

Structure	Advantages	Disadvantages	Suitable For
Single-level	Simple, fast	Large table size	Small address space
Hierarchical	Saves on unused regions	Too many levels for 64-bit	32-bit systems
Hashed	Handles large address spaces	Hash collision overhead	64-bit, sparse spaces
Inverted	Based on physical memory, space-saving	Slow search, difficult shared pages	Large physical memory

Part 7

Swapping

Standard Swapping

Exchanging an entire process between memory and disk (backing store)

```
Main Memory                Backing Store (Disk)
+————————————+
|     OS     |
+————————————+      swap out
| Process 1  | ————————————→  +————————————+
+————————————+                | Process 1  |
| Process 2  |                +————————————+
+————————————+      swap in   | Process 3  |
|  (free)    | ←————————————  +————————————+
+————————————+
```

- **Swap out**: Move entire process from memory -> disk
- **Swap in**: Restore entire process from disk -> memory
- Disadvantage: Transfer time is very large -> rarely used in modern OSes

Swapping with Paging (Modern OS)

A method that swaps out/in at the **page level** (the modern OS standard)

- Only **needed pages** are exchanged, not the entire process
- Much more efficient and faster
- **Page out**: Memory -> backing store
- **Page in**: Backing store -> memory

```
Process A                Main Memory          Backing Store
pages: a,b,c,d,e    +---+---+---+---+---+
                     | b | c | e |   |   |   a, d → page out
Process B            +---+---+---+---+---+
pages: f,g,h,i,j    | f | h | j |   |   |   g, i → page out
                     +---+---+---+---+---+
```

- Covered in detail later in **Virtual Memory** (Ch 10)

Swapping on Mobile Systems

Swapping on mobile systems

- Most mobile OSes **do not support** swapping
- Reasons:
  - **Limited write cycles** of flash memory (lifespan concern)
  - **Low throughput** between flash memory and main memory
  - **Storage space** constraints

iOS: Requests apps to voluntarily release memory when memory is low; terminates apps that do not comply

Android: Saves **app state to flash** before terminating processes when memory is low -> enables fast restart

Part 8

Real-World Architectures

IA-32 Architecture (32-bit x86)

Segmentation + Paging combined approach

```
CPU → Logical    → Segmentation → Linear    → Paging → Physical
      Address       Unit           Address     Unit     Address

Segmentation:
+———————+———————+
| selector | offset   |    → references descriptor table
| (16 bit) | (32 bit) |    → base + offset = linear address
+———————+———————+

Two-Level Paging (4KB page):
+———————+———————+———————+
|    p1    |    p2    |    d     |
| (10 bit) | (10 bit) | (12 bit) |
+———————+———————+———————+
CR3 → Page Directory → Page Table → 4KB Page
```

- When using 4MB pages: Page Directory directly points to a 4MB frame

IA-32 PAE (Page Address Extension)

An extension that allows 32-bit processors to access more than 4GB of physical memory

```
PAE disabled (2-level):
+------+------+------+
|  p1  |  p2  |  d   |     → Max 4GB physical memory
| (10) | (10) | (12) |
+------+------+------+

PAE enabled (3-level):
+----+------+------+------+
|pdpt|  pd  |  pt  |  d   |  → Max 64GB physical memory
| (2)| (9)  | (9)  | (12) |
+----+------+------+------+

CR3 → Page Dir Pointer Table → Page Dir → Page Table → 4KB Page
```

- Page table entry expanded from 32-bit to **64-bit**
- Base address: 20-bit -> **24-bit** (+ 12-bit offset = 36-bit physical address)
- Linux and macOS support PAE; 32-bit Windows desktop still limited to 4GB

x86-64 Architecture

64-bit extension architecture (based on AMD64, also adopted by Intel)

```
x86-64 Linear Address (48-bit virtual → 52-bit physical):

+————————+———————+———————+———————+———————+———————+
| unused | PML4  | PDPT  |  PD   |  PT   | offset|
| (16)   | (9)   | (9)   | (9)   | (9)   | (12)  |
+————————+———————+———————+———————+———————+———————+
63    48 47   39 38  30 29  21 20  12 11    0

4-level paging hierarchy
```

- **48-bit** virtual address space = 256TB
- **52-bit** physical address (PAE extension) = 4PB
- Page size: **4KB**, **2MB**, **1GB**
- Segmentation effectively disabled

ARMv8 Architecture (64-bit ARM)

The most widely used 64-bit architecture in mobile/embedded systems

```
ARMv8 4KB Translation Granule:

+────────+──────+──────+──────+──────+──────+
| unused |  L0  |  L1  |  L2  |  L3  | offset|
| (16)   | (9)  | (9)  | (9)  | (9)  | (12)  |
+────────+──────+──────+──────+──────+──────+
63    48 47   39 38  30 29  21 20  12 11    0

TTBR → L0 Table → L1 Table → L2 Table → L3 Table → 4KB Page
                      |            |
                  1GB region   2MB region (shortcut possible)
```

Translation Granule	Page Size	Region Size
4 KB	4 KB	2 MB, 1 GB
16 KB	16 KB	32 MB

Architecture Comparison Summary

Feature	IA-32	x86-64	ARMv8
Address size	32-bit	48-bit virtual	48-bit virtual
Paging levels	2-level (3 with PAE)	4-level	Up to 4-level
Page sizes	4KB, 4MB	4KB, 2MB, 1GB	4KB, 16KB, 64KB
Segmentation	Used	Effectively disabled	None
Physical memory	4GB (64GB w/ PAE)	4PB (52-bit)	Up to 48-bit
TLB	Present	Multi-level	2-level

Lab

Memory Allocation Simulator

Lab – Memory Allocation Simulator (1)

Comparing First-fit, Best-fit, and Worst-fit

Objective: Manage a memory block list and compare three allocation algorithms

Implementation requirements:

- Represent memory as a block list (start address, size, status)
- Process allocation request: Allocate using each of the three algorithms
- Process release request: Coalesce adjacent free blocks
- Output: Memory state, allocation success/failure, external fragmentation size

Lab – Memory Allocation Simulator (2)

Data structure design:

```
typedef struct Block {
    int start;        // Start address
    int size;         // Block size
    int allocated;    // 0: free, 1: allocated
    char process[10]; // Process name
    struct Block* next;
} Block;
```

Key functions:

```
Block* first_fit(Block* head, int size);   // First sufficient hole
Block* best_fit(Block* head, int size);    // Smallest sufficient hole
Block* worst_fit(Block* head, int size);   // Largest hole
void release(Block* head, char* process);  // Release + coalesce
void print_memory(Block* head);            // Print memory state
```

Lab – Memory Allocation Simulator (3)

Input and output example

```
Input:
  Memory size: 1024
  Allocate P1: 200
  Allocate P2: 350
  Allocate P3: 150
  Release P2
  Allocate P4: 300

Output (First-fit):
  [  0- 199] P1 (200)
  [200- 549] FREE (350)    ← P2 released
  [550- 699] P3 (150)
  [700-1023] FREE (324)

  Allocate P4(300) → First-fit: [200-499] P4
  [  0- 199] P1 (200)
  [200- 499] P4 (300)
  [500- 549] FREE (50)     ← remaining hole
  [550- 699] P3 (150)
  [700-1023] FREE (324)

  External fragmentation: 50 + 324 = 374 bytes
```

Summary

Key points from this week

- **Address Binding**: compile time / load time / execution time
- **Logical vs Physical address**: MMU performs the translation
- **Contiguous Allocation**: First-fit, Best-fit, Worst-fit
- **Fragmentation**: External (lack of contiguous space) / Internal (waste within allocation unit)
- **Paging**: page -> frame mapping, page table, TLB for performance improvement
- **Page Table Structure**: Hierarchical / Hashed / Inverted
- **Swapping**: standard (entire process) vs paging (page-level)
- **Real-World**: IA-32 (segmentation+paging), x86-64 (4-level), ARMv8 (4-level)

Next Week Preview

Week 12: Virtual Memory (Ch 10)

- Demand Paging
- Copy-on-Write
- Page Replacement Algorithms (FIFO, Optimal, LRU)
- Allocation of Frames
- Thrashing
- Memory-Mapped Files